AST-Compressed Code Graphs for Gemma 4 Agents: What a Small Local Model Gains from a Budgeted Repository Map

Muhammad Maroof
Independent researcher, Karachi, Pakistan maroofiums@gmail.com
Code and data: https://github.com/maroofiums/astgraph-gemma

Gemma 4 Developer Agent Paper Track

Abstract

Running a coding agent on a single consumer GPU means the repository cannot be pasted into the prompt, and a small model gets lost when it is. We build a repository map from tree-sitter ASTs (signatures, line ranges, optionally call edges and control-flow counts), rank symbols against the issue text, and fill a fixed token budget by score per token. We test it with Gemma 4 E4B (4-bit, one T4) on SWE-bench Lite issue localization, 150 test instances, with the ranker chosen on a separate dev split without any LLM calls. At a 2k-token budget a plain signature map reaches 0.527 file Acc@1 against 0.380 for function-snippet RAG and 0.280 for raw top-k files, and doubles BM25 file ranking (0.267). Our richer variant, which adds call edges and control-flow counts, did *not* help: it is 0.04 lower on file accuracy (not significant) and 0.09 lower on function accuracy ($p_{\text{Holm}} = 0.002$). At function level, snippet RAG matches the signature map, and a two-stage prompt (map first, then candidate files) lifts function accuracy over the full variant (0.042 to 0.229 on the ablation subset) but not significantly over the plain signature map (0.188; difference $+0.042$, $p = 0.54$). Memory measurements show that for this model the KV cache is a small part of what a long prompt costs; logits and activations dominate, and chunked prefill with last-token logits is what lets an 8k prompt be prefilled on a T4 (measured for prefill only). We also report the failure analysis, the negative results, and how the index can be used either as a prompt layer or as a set of tools.

1 Introduction

Most agent work on repository-level tasks assumes a large hosted model. If the model has to run locally, for privacy or cost, two problems show up together. The prompt budget is small, and the model is easy to distract. Pasting the files that a keyword search ranks highest spends the budget on method bodies the model does not need yet, and a model of Gemma 4 E4B's size (about four billion effective parameters) has little slack to ignore irrelevant text.

Repository maps are the usual answer. Aider's map lists files with the signatures of their most referenced symbols [3]; RepoGraph and CodexGraph go further and put a graph database behind the agent [4, 5]. What is less clear is which parts of such a representation matter for a small model, how the ranking interacts with the budget, and what the memory side looks like on 15 GB of VRAM. This paper is a measured answer to those three questions, on one model and one task.

Task. Given an issue and a repository checkout, name up to three files and one function where the fix belongs. Formally, for an issue text q and a repository with file set $\mathscr{F}$, the model returns an ordered list $\hat{F} = (\hat{f}_1, \hat{f}_2, \hat{f}_3) \subseteq \mathscr{F}$ and one symbol $\hat{s}$; the reference patch defines the gold files $F^\star \subseteq \mathscr{F}$ and the set $L^\star$ of touched lines. This is the localization step of systems like Agentless [2]; we do not generate patches.

Contributions.

- An open implementation (`astgraph`) that builds a budgeted map from tree-sitter, with four rankers and a guarantee that the rendered text fits the budget.

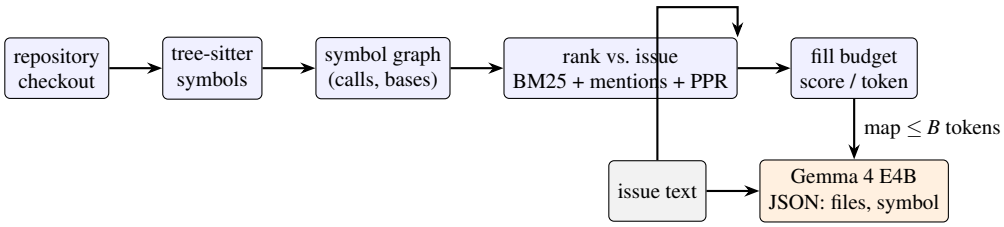


Figure 1: Pipeline. The map is rebuilt per issue from a cached parse; only the ranking and the fill step depend on the issue.

- A benchmark protocol with a dev split used only to pick the ranker (no LLM), a disjoint test split, bootstrap intervals and Holm-corrected paired tests.
- Results that are partly negative: signatures carry the gain; call edges and control-flow counts, as we serialize them, do not.
- A memory analysis of Gemma 4 E4B on a T4 that separates weights, KV cache and logits, and shows where the budget actually goes.

2 Method

Symbols and graph. For every `.py` file we parse with tree-sitter [9] and record, for each class, method and function, its qualified name, one-line signature (with annotations), line range, base classes, the names it calls, and counts of `if/for/while/try/with/match` nodes. Calls are resolved by name only, so an edge $a \to b$ means "a calls something called b", weighted by $1/\#\text{candidates}$. Writing V for the set of symbols, $\text{callees}(a)$ for the names called inside a, and $C(n) = \{v \in V : \text{name}(v) = n\}$ for the candidates of a name n, the edge weights are

$$w(a \to v) = \sum_{n \in \text{callees}(a)} \frac{\mathbf{1}[v \in C(n)]}{|C(n)|}, \tag{1}$$

so every call name spreads a total weight of at most one over its candidates. This is crude; there is no type inference. Files are keyed by SHA-1, so a re-index after a commit parses only changed files.

Ranking. Each symbol v becomes a small document d_v (identifier words from its qualified name, its file path, and its callees). We score documents against the issue q with BM25 [6],

$$\begin{aligned} \text{bm25}(q,d) &= \sum_{t \in q} \text{idf}(t)\, \frac{f_{t,d}\,(k_1+1)}{f_{t,d} + k_1\big(1 - b + b\,|d|/\overline{|d|}\big)}, \\ \text{idf}(t) &= \ln\!\Big(1 + \frac{|V| - \text{df}_t + \frac{1}{2}}{\text{df}_t + \frac{1}{2}}\Big), \end{aligned} \tag{2}$$

where $f_{t,d}$ counts term t in d, df_t is the number of documents containing t, and k_1, b are the usual BM25 constants. We compare four rankers. **bm25** uses $s_{\text{bm25}}(v) = \text{bm25}(q, d_v)$. **bm25m** adds a boost when the issue mentions the file path (including paths in tracebacks) or the symbol name (backticked, `snake_case` or `CamelCase`):

$$s_{\text{m}}(v) = \text{bm25}(q,d_v) + \delta_{\text{path}}\,\mathbf{1}[\text{path}(v) \text{ mentioned}] + \delta_{\text{name}}\,\mathbf{1}[\text{name}(v) \text{ mentioned}]\,. \tag{3}$$

ppr is personalised PageRank [7] on the undirected graph with weights $\tilde{w}(u,v) = w(u \to v) + w(v \to u)$, seeded with the normalised BM25 scores $p_v = \text{bm25}(q,d_v) / \sum_u \text{bm25}(q,d_u)$:

$$r = \alpha\, P^{\top} r + (1-\alpha)\, p, \qquad P_{uv} = \frac{\tilde{w}(u,v)}{\sum_x \tilde{w}(u,x)}, \qquad \alpha = 0.5. \tag{4}$$

hybrid mixes the two, both max-normalised:

$$s(v) = 0.7\,\bar{s}_{\text{m}}(v) + 0.3\,\bar{r}(v), \qquad \bar{x}(v) = \frac{x(v)}{\max_u x(u)}. \tag{5}$$

Budgeted rendering. Candidates are the top 3000 symbols V_{3000} by s. Each has a cost τ: the length of its rendered line in tokens, plus the file path the first time its file appears. For a symbol v in file $\pi(v)$, given the set S already chosen,

$$\tau(v \mid S) = \ell(v) + \ell\big(\pi(v)\big)\,\mathbf{1}[\pi(v) \notin \pi(S)]\,, \tag{6}$$

where $\ell(\cdot)$ is a token length. The map is a greedy solution of the knapsack-style problem

$$\max_{S \subseteq V_{3000}} \sum_{v \in S} s(v) \qquad \text{s.t.} \qquad \sum_{v \in S} \tau\big(v \mid S_{<v}\big) \le B, \tag{7}$$

where $S_{<v}$ are the symbols added before v: we take symbols in decreasing order of score per token, $s(v)/\tau(v)$, and keep each one while it fits. This is not optimal in general. Files are listed by their best score, $s(f) = \max_{v \in f} s(v)$. The final text is re-counted and trimmed if the greedy estimate was off, so that $|\mathrm{tok}(\mathrm{render}(S))| \le B$ always holds and the budget is a hard limit. A row looks like `m def parse_date(self, s) -> str L41-58`; the optional suffixes are `>callee1,callee2` and `#if2 for1`.

Prompt. The system prompt asks for JSON only: up to three files and one qualified symbol. The map goes first, the issue (truncated to 1024 tokens for every method) after it, and decoding is greedy with at most 200 new tokens. Predictions are parsed leniently; a symbol is cleaned of `def`, file prefixes and argument lists.

3 Experimental setup

Model and hardware. `google/gemma-4-E4B-it` [10] through Hugging Face Transformers, bitsandbytes NF4 [8] with fp16 compute, one NVIDIA T4 (14.56 GiB usable), SDPA attention. All latency numbers are wall-clock on this setup and include repository ingest. Ingest used a content-hash parse cache shared across instances, so it understates cold-start cost, and the time spent scoring symbols and building the baselines' retrieval indices is not included.

Data. SWE-bench Lite [1] (300 test instances, 12 repositories), sorted by id and shuffled with seed 0. The last 60 form the *dev* split; the first 150 of the remainder are the *test* split (instances are disjoint, repositories overlap). Ablations and the budget sweep use the first 60 test instances. Gold files and gold functions come from the reference patch: a function is a hit if the predicted symbol matches (by qualified or leaf name) any symbol enclosing a touched line in the predicted file. Function accuracy is defined only when every touched line sits inside a function or class; this holds for 119 of the 150 test instances and 48 of the first 60.

Methods at equal budget $B = 2048$. *issue only*; *BM25 files* (no LLM, rank files by BM25 over their text); *raw top-k*: the top ten BM25 files pasted whole and cut at B; *snippet RAG*: BM25 over function bodies (first 40 lines each), best snippets until B; SIGMAP: our map with signatures only (an approximate re-implementation of the Aider idea, not Aider itself); OURS: signatures plus callees plus control-flow counts. Both graph methods use the ranker chosen on dev.

Metrics and statistics. We report File Acc@1 and @3, function Acc@1, prompt tokens, prefill time and end-to-end latency. For instance i let $\hat{f}_{i,1}, \hat{f}_{i,2}, \hat{f}_{i,3}$ be the predicted files, $F_i^\star$ the gold files, $\hat{s}_i$ the predicted symbol, and $\mathrm{Enc}_i(f)$ the symbols of file f that enclose a touched line in $L_i^\star$. With n instances and I_f the subset on which function accuracy is defined,

$$\mathrm{File@}k = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\big[\{\hat{f}_{i,1}, \ldots, \hat{f}_{i,k}\} \cap F_i^\star \neq \emptyset\big]\,, \qquad \mathrm{Func@1} = \frac{1}{|I_\mathrm{f}|}\sum_{i \in I_\mathrm{f}} \mathbf{1}\big[\exists\, v \in \mathrm{Enc}_i(\hat{f}_{i,1}) :\ \hat{s}_i \sim v\big]\,, \tag{8}$$

where $\sim$ is a match by qualified or leaf name. For a per-instance hit indicator $h_i^A \in \{0,1\}$ of method A, the point estimate is $\hat{p}_A = \frac{1}{n}\sum_i h_i^A$, and the 95% interval is the 2.5th to 97.5th percentile of $\hat{p}_A$ over 10,000 bootstrap resamples of the instances. Comparisons use the paired difference $\hat{\Delta}_{AB} = \frac{1}{n}\sum_i \big(h_i^A - h_i^B\big)$, resampling instances jointly. Within each report of m comparisons the sorted p-values $p_{(1)} \le \cdots \le p_{(m)}$ are Holm-adjusted,

$$p_{(j)}^{\mathrm{Holm}} = \min\Big\{1,\ \max_{k \le j} (m-k+1)\, p_{(k)}\Big\}. \tag{9}$$

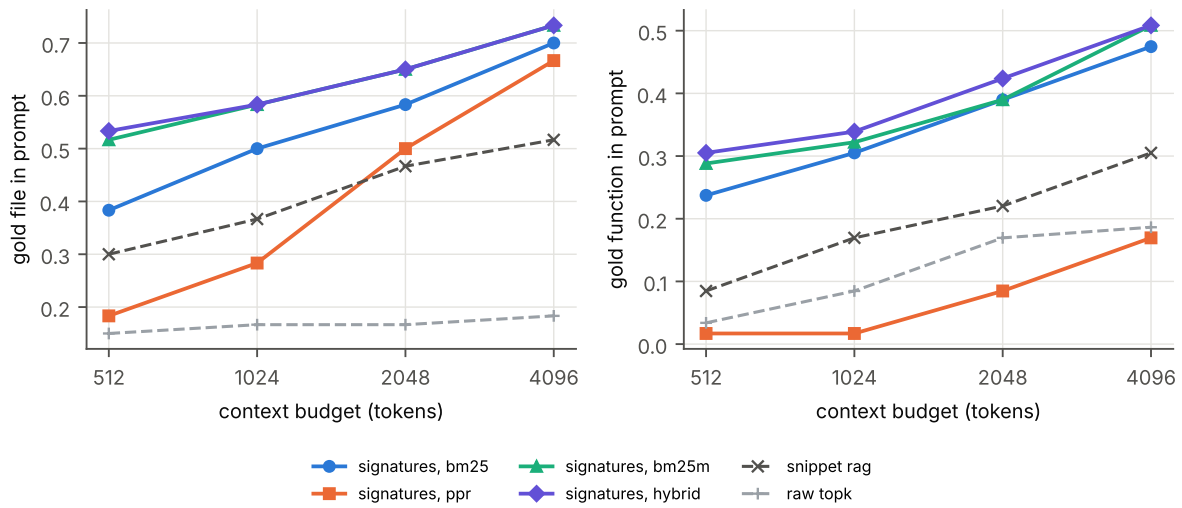


Figure 2: Dev split ($n = 60$, no LLM): fraction of prompts that contain the gold file (left) and the gold function definition (right), by context budget. Signature-map rankers are solid; text baselines dashed.

Table 1: Test split, $n = 150$ (function accuracy on 119 instances), $B = 2048$. Brackets are 95% bootstrap intervals. Latency is end to end on a T4 including ingest.

Method	File@1	File@3	Func@1	ctx tok	prompt tok	p50 s	p95 s
issue only	0.207 [.147, .273]	0.253	0.109 [.059, .168]	0	507	12.3	23.2
BM25 files (no LLM)	0.267 [.200, .340]	0.460	–	0	0	4.7	15.9
raw top-k	0.280 [.213, .353]	0.293	0.109 [.059, .168]	2037	2548	13.4	25.1
snippet RAG	0.380 [.300, .460]	0.447	0.227 [.160, .303]	2026	2558	13.8	26.2
OURS (edges + flow)	0.487 [.407, .567]	0.627	0.143 [.084, .210]	2034	2592	17.6	28.2
SIGMAP (signatures)	**0.527** [.447, .607]	**0.647**	**0.235** [.160, .311]	2037	2595	17.0	29.8

The normal-approximation half-width of a 95% interval is $1.96\sqrt{\hat{p}(1-\hat{p})/n}$, which for $\hat{p} = 0.5$ is about 0.08 at $n = 150$ and 0.13 at $n = 60$, so differences below roughly 0.06 should be read as noise.

4 Results

4.1 Choosing the ranker on dev, without the LLM

For each ranker and budget we measured how often the prompt *contains* the gold file and the gold function definition (Figure 2). Hybrid is on or above every other ranker at every budget; bm25m is close. PPR alone is poor at small budgets (gold file in prompt 0.18 at 512 tokens, 0.50 at 2048, against 0.65 for hybrid), because it spreads mass over graph hubs and ignores what the issue actually names. Let $\mathrm{Cov}_{\mathrm{file}}(\rho, B) = \frac{1}{n}\sum_i \mathbf{1}\left[\text{gold file of } i \text{ appears in } M_i^{\rho,B}\right]$ be the fraction of prompts that contain the gold file under ranker ρ and budget B, and $\mathrm{Cov}_{\mathrm{func}}$ the same for the gold function definition. We chose the ranker that maximises

$$J(\rho) = \frac{1}{2} \sum_{B \in \{1024,\, 2048\}} \left[\mathrm{Cov}_{\mathrm{file}}(\rho, B) + \mathrm{Cov}_{\mathrm{func}}(\rho, B)\right] \in [0, 2], \tag{10}$$

which picks hybrid ($J = 0.998$, against 0.973, 0.889 and 0.443 for bm25m, bm25 and ppr). Structure also beats text at equal budget: at 2048 tokens, 0.65 of gold files are in a signature map, 0.47 in snippet RAG and 0.17 in raw top-k. For comparison, BM25 over whole files has file recall of 0.15, 0.43 and 0.67 at ranks 1, 3 and 10.

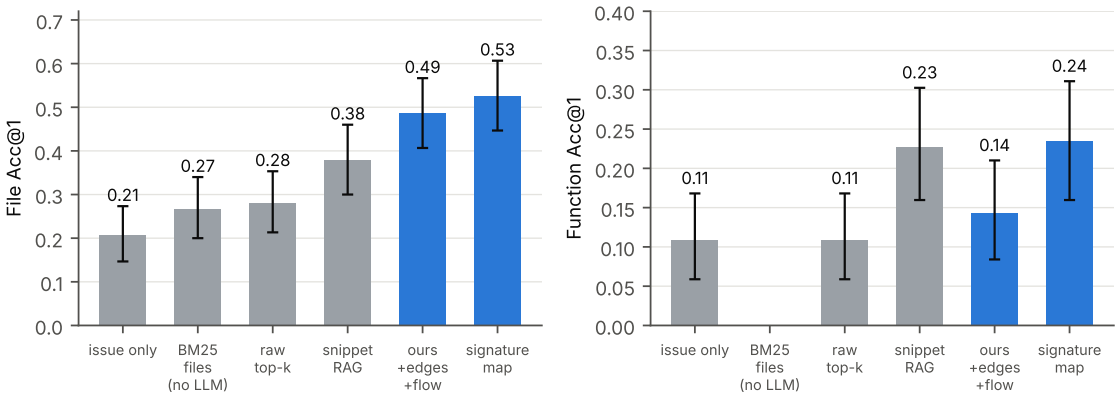


Figure 3: Table 1 as bars with 95% intervals. Blue: graph methods.

Table 2: Ablations on the first 60 test instances (function accuracy on 48). Differences are against SIGMAP in the top block and against OURS in the bottom block.

Variant	File@1	File@3	Func@1	prompt tok	Δfile / Δfunc (p_{Holm})
SIGMAP	0.500	0.650	0.188	2614	reference
+ edges	0.467	0.667	0.125	2614	−.033 (.78) / −.062 (.34)
+ control flow	0.533	0.683	0.208	2614	+.033 (.88) / +.021 (.88)
OURS (both)	0.417	0.617	0.042	2612	−.083 (.08) / −.146 (<.001)
SIGMAP with PPR ranker	0.333	0.483	0.062	2612	−.167 (.20) / −.125 (.29)
OURS	0.417	0.617	0.042	2612	reference
JSON rows instead of compact	0.400	0.633	0.062	2597	−.017 (1.0) / +.021 (1.0)
files alphabetical	0.550	0.650	0.125	2605	+.133 (.20) / +.083 (.60)
files shuffled	0.467	0.650	0.146	2605	+.050 (1.0) / +.104 (.053)
two-stage	0.467	0.617	0.229	3803	+.050 (.95) / +.188 (.003)

4.2 Main comparison

Table 1 and Figure 3 show the result we care about most: at the same 2k budget, a signature map locates the right file for 53% of issues, snippet RAG for 38% and raw files for 28%. Against SIGMAP, snippet RAG is 0.147 lower on file Acc@1 (95% CI −0.233 to −0.060, $p_{\text{Holm}} = 0.004$), raw top-k 0.247 lower ($p_{\text{Holm}} < 0.001$). The model with no context at all gets 0.207, so the 4B model is not guessing from the issue alone, but BM25 over files, with no LLM, already beats it on file@1 (0.267). Relative to BM25 file ranking the signature map is better by a factor of $0.527/0.267 \approx 1.97$ on File@1, which is the "doubling" quoted in the abstract.

Two things temper this. First, OURS is also well ahead of snippet RAG on files (+0.107, paired $p = 0.023$, unadjusted, post hoc) but *below* SIGMAP: −0.040 on file Acc@1 (CI −0.087 to +0.007, $p_{\text{Holm}} = 0.26$) and −0.092 on function Acc@1 (CI −0.151 to −0.042, $p_{\text{Holm}} = 0.002$). Second, at function level snippet RAG is as good as the signature map (0.227 against 0.235, difference −0.008, $p = 0.93$). A signature map shows names, not code; picking the function inside a file is a different problem from picking the file. Latency is 3–4 s higher for the graph methods at p50, mostly prefill on a longer prompt (about 3.0 s against 2.9 s median prefill) plus a longer answer (60 against 46 generated tokens).

4.3 Ablations: edges, control flow, format, order

Neither call edges nor control-flow counts helps on its own, and together they hurt function accuracy (Table 2). The tokens they take are not free: with the budget fixed, every callee list pushes some other symbol out of the map, and the function-level gold is more often missing (dev coverage of the gold function at 2048 tokens drops from 0.42 for signatures to 0.29 for OURS). Whether the model also reads the extra columns badly we cannot say from this data. Control flow alone is nominally positive but far inside the noise.

Format and ordering effects are all unresolved at $n = 60$. We note only that alphabetical and shuffled file order scored at least as well as relevance order, so we cannot claim that listing the most relevant file first helps this model.

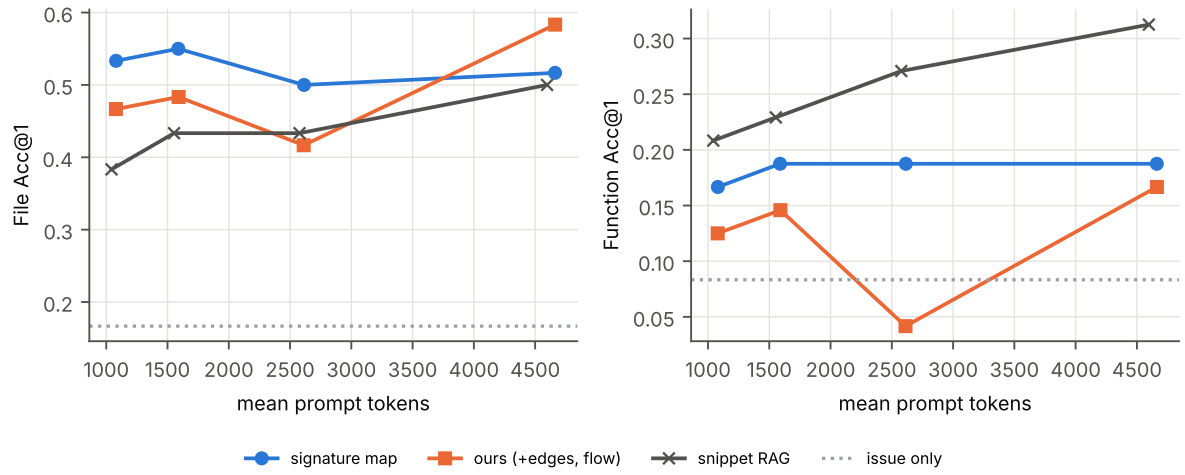


Figure 4: Accuracy against mean prompt tokens on the first 60 test instances. Dotted: issue only (0.167 file, 0.083 function).

The old PPR-only ranker is clearly worse in point estimate (0.333 against 0.500), consistent with the dev coverage numbers, but the Holm-adjusted test does not reject.

The two-stage variant is the only change that moves function accuracy relative to OURS, but the fair comparison is SIGMAP. Stage one shows the model only file names and symbol names (half the budget) and asks for candidate files; stage two re-renders the full map restricted to those files. With $M_{B/2}^{\text{names}}$ the stage-one view and $M_B(\cdot \mid C)$ the map of Section 2 restricted to a file set C,

$$C = \text{LLM}\big(q, M_{B/2}^{\text{names}}\big), \qquad (\hat{F}, \hat{s}) = \text{LLM}\big(q, M_B(\cdot \mid C)\big). \tag{11}$$

Function accuracy rises from 0.042 to 0.229 ($+0.188$, $p_{\text{Holm}} = 0.003$) with file accuracy unchanged, at the cost of 1191 more prompt tokens and about 4.5 s more median latency (23.1 s against 18.6 s). Against SIGMAP (0.188 on the same 48 instances) the difference is $+0.042$ on function accuracy (95% CI -0.062 to $+0.146$, $p = 0.54$) and -0.033 on file accuracy, so we cannot say that a second stage beats a single-stage signature map; OURS is an unusually low reference on this subset (0.042 here, 0.143 on all 150 test instances).

4.4 Budget sweep

Figure 4 shows file accuracy for SIGMAP is flat from about 1.1k to 4.7k prompt tokens (0.533, 0.550, 0.500, 0.517), so on this model more map does not give more localization. OURS is non-monotone (0.467, 0.483, 0.417, 0.583), which with $n = 60$ and intervals of roughly ± 0.12 we read as noise rather than a trend. Snippet RAG improves steadily with budget, on files (0.383 to 0.500) and especially on functions (0.208 to 0.313), and has the highest function-level point estimate at every budget (the intervals overlap; at 4096 tokens the gap to SIGMAP is $+0.125$, $p = 0.094$). If the prompt is cheap and the goal is the function, bodies beat signatures; if the goal is the file and the prompt must be small, signatures win, and 512 tokens of map (about 1.1k prompt tokens) already gives most of the benefit.

4.5 Where the failures are

On the first 60 test instances the gold file is in the prompt for 78% of issues with OURS (47/60) and 80% with SIGMAP (48/60). When it is absent the model never gets it right (0/13 and 0/12). When it is present, the model picks it first 53% and 62% of the time. Of the 35 failures with OURS (30 with SIGMAP), 13 (12) are ranking misses; the remaining 22 (18) are cases where the right file was visible and the model chose a plausible neighbour: `django/db/models/query.py` for `django/db/models/sql/query.py`, `Dataset` for `DataArray` in xarray, a test file for the module under test. Per repository the picture is uneven (for example scikit-learn 0.79 and Django 0.64 for SIGMAP, Flask 0.00 on a single instance); the repositories with few test instances are too small to say anything about.

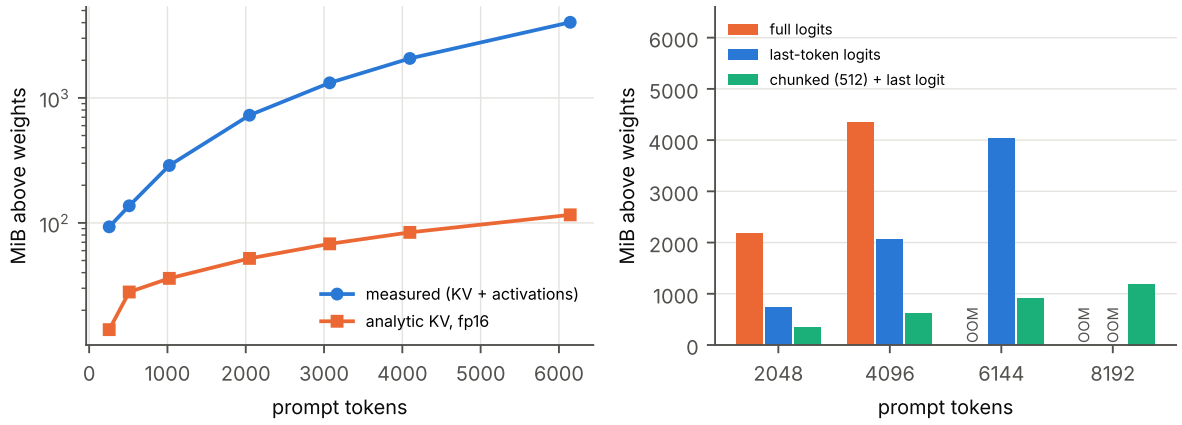


Figure 5: Left: memory above the weights during prefill, measured against the analytic fp16 KV cache (only the 24 layers that own a cache, sliding layers capped at 512 tokens). Right: peak extra memory by prefill strategy.

4.6 Memory and prefill on a T4

The checkpoint is not as small as "4-bit" suggests. Resident weights are 9081 MiB. Of the 8802 MiB of parameters, 5376 MiB are per-layer embeddings and 1280 MiB the token embedding, both left in bf16; only the 2146 MiB of compute weights are quantized. The model has 42 layers (35 sliding-window with window 512, 7 full attention), 2 KV heads, head dimension 256 (512 in the full layers), and 18 layers share KV with earlier ones. Its analytic fp16 KV cache is therefore tiny: 14 MiB at 256 tokens and 116 MiB at 6144. With $n_{kv} = 2$ heads and $\beta = 2$ bytes per element, and, among the 24 layers that own a cache, $L_\mathrm{f} = 4$ full-attention layers (head dimension $d_\mathrm{f} = 512$) and $L_\mathrm{s} = 20$ sliding layers ($d_\mathrm{s} = 256$, window $w = 512$), the cache for n tokens is

$$M_{\mathrm{KV}}(n) = 2\,n_{kv}\,\beta\left[L_\mathrm{f}\,d_\mathrm{f}\,n + L_\mathrm{s}\,d_\mathrm{s}\min(n,w)\right] = \begin{cases} 56\ \mathrm{KiB}\cdot n, & n \le 512, \\ 16\ \mathrm{KiB}\cdot n + 20\ \mathrm{MiB}, & n \ge 512, \end{cases} \tag{12}$$

which reproduces both numbers above.

Measured memory above the weights is far larger (Figure 5, left): 93 MiB at 256 tokens, 727 MiB at 2048, 4026 MiB at 6144. Writing $\Delta M(n)$ for the measured memory above the weights, the KV share $M_{\mathrm{KV}}(n)/\Delta M(n)$ is $116/4026 \approx 3\%$ at 6144 tokens and at most about 20% at any length. The rest is activations and logits, and the vocabulary of $V = 262{,}144$ makes logits expensive: the logits for m positions take $mV\beta$ bytes, which in fp16 is 2048 MiB for $m = 4096$ positions against 512 KiB for the last position alone, the same order as the $4340 - 2069 = 2271$ MiB saved below. Computing logits for every position at 4096 tokens costs 4340 MiB above weights; asking only for the last position costs 2069 MiB; prefilling in chunks of 512 with last-token logits costs 624 MiB. At 6144 tokens, full logits run out of memory, and at 8192 only the chunked path runs (1191 MiB, 15.4 s). These probes measure prefill memory on random token ids; we did not check answer quality at 8k tokens or the memory of decoding after a chunked prefill. Prefill time grows faster than linearly: 0.51 s at 512 tokens, 2.4 s at 2048, 9.0 s at 4096. Writing $T(n) \propto n^\gamma$, the local exponent is $\gamma = \ln(2.4/0.51)/\ln 4 \approx 1.1$ between 512 and 2048 tokens and $\gamma = \log_2(9.0/2.4) \approx 1.9$ between 2048 and 4096.

Two conclusions follow for local deployment. A small prompt matters, but the thing it saves is mostly prefill time and activation memory, not KV cache; and chunked prefill with last-token logits is worth turning on before touching the context budget. We did not test KV-cache quantization or other quantization schemes.

4.7 Compression ratio

Over ten test instances, the full symbol graph of a repository (all symbols, signatures, edges, control flow) is on average 4.8× smaller than its Python source, with source size estimated at 3.5 characters per token. For one instance let T_{src} be the source size in tokens (characters/3.5), T_{graph} the size of the full symbol graph and $T_{\mathrm{map}} \le B$ the size of the budgeted map. Then

$$\rho_{\mathrm{total}} = \frac{T_{\mathrm{src}}}{T_{\mathrm{map}}} = \underbrace{\frac{T_{\mathrm{src}}}{T_{\mathrm{graph}}}}_{\rho_{\mathrm{AST}}} \cdot \underbrace{\frac{T_{\mathrm{graph}}}{T_{\mathrm{map}}}}_{\rho_{\mathrm{select}}}. \tag{13}$$

After ranking and budgeting to 2k tokens, the ratio is about 2300× (range 500 to 4200). The first number is ρ_{AST}, what AST compression buys on its own; the second is ρ_{total}, which is mostly ρ_{select}. We report both because the large one should not be credited to the parser.

5 Using the index in a project

The experiments use one configuration: the map is built before the call and prepended to the prompt. Two other uses follow from the results, and we implemented both in the released package.

Prompt layer. A function `RepoIndex.context(issue, budget)` returns the map; the caller prepends it to whatever prompt it already has. This works with any runtime, since it only needs a string, and is the setup measured above. The defaults are the best configuration found: signatures only, hybrid ranker.

Tools. The same index can be offered to the model as function-calling tools: `repo_map`, `find_symbol`, `read_symbol`, `callers`, `callees`, `outline`; the package also wraps them as an MCP server. This follows from two findings. Edges did not help when serialized into the prompt, but they are useful to *navigate*, so `callers` and `callees` make the graph available on demand without spending budget. And function-level accuracy only moved when the model was given a second, narrower view, which is what a `read_symbol` call provides. We have unit-tested the tool layer and the agent loop against a scripted model; we have **not** yet measured it with Gemma 4, so this is a hypothesis, not a result. The experiment is described in the repository (`docs/integration.md`).

6 Limitations

- One model, one quantization, greedy decoding, one run. We have no variance across seeds or across Gemma sizes.
- Localization only. We do not generate or test patches, so these numbers say nothing about resolve rate.
- Intervals are wide. With 150 instances (60 for ablations) many comparisons are inconclusive; the ones we call results have Holm-adjusted $p < 0.01$.
- Python only. The parser is tree-sitter, so other languages need a grammar and the symbol rules, but we have not done it.
- Name-based call resolution produces false edges for common names; the "edges hurt" result applies to this resolver and to how we serialize edges.
- SIGMAP is our own approximation of an Aider-style map. A comparison with the real tool, and with agentic search, is future work.
- Dev and test splits share repositories, and we do not know whether SWE-bench Lite is in Gemma 4's training data.
- Function accuracy is lenient (leaf-name match) and defined on a subset of instances.
- End-to-end latency includes ingest with a warm parse cache and excludes symbol-scoring time (Section 3), so it is indicative rather than a cold-start figure.

7 Conclusion

For a 4B-effective local model with a 2k-token context budget, the useful part of an AST map is the ranked list of files and signatures; that alone doubles BM25 file ranking and beats code snippets and raw files on file localization. The additions we expected to help, call edges and control-flow counts in the prompt, did not. Function-level localization needs code bodies or a second stage, and snippet RAG is already as good as the map there. On a T4 the memory story is about logits and activations rather than KV cache. The next experiments are the tool-mode agent with Gemma 4, a hybrid that uses the map to pick files and snippet retrieval inside them, and more instances with several seeds.

References

[1] C. E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.

[2] C. S. Xia et al. Agentless: Demystifying LLM-based software engineering agents. 2024. arXiv:2407.01489.

[3] P. Gauthier. Building a better repository map with tree sitter. Aider documentation, https://aider.chat/docs/repomap.html.

[4] S. Ouyang et al. RepoGraph: Enhancing AI software engineering with repository-level code graph. 2024. arXiv:2410.14684.

[5] X. Liu et al. CodexGraph: Bridging large language models and code repositories via code graph databases. 2024. arXiv:2408.03910.

[6] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval 3(4), 2009.

[7] L. Page, S. Brin, R. Motwani, T. Winograd. The PageRank citation ranking: bringing order to the web. Stanford InfoLab technical report, 1999.

[8] T. Dettmers et al. QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023. arXiv:2305.14314.

[9] tree-sitter, https://tree-sitter.github.io/.

[10] Gemma 4 E4B instruction-tuned, `google/gemma-4-E4B-it`, Hugging Face model card.